

Análisis del Proyecto

App de Campeonatos Deportivos UPSA (Flutter + Firebase)

Documento generado automáticamente a partir de una lectura completa del código fuente real del proyecto (carpeta `lib/`). No se modificó ningún archivo del proyecto para producir este análisis.

Nombre técnico del paquete: **futsal** | Título visible: **UPSA Campeonatos**

Framework: Flutter (Dart, SDK ^3.10.0) | Backend: Firebase (Auth + Cloud Firestore)

Sección 1 — Estructura del proyecto

1.1 Árbol de carpetas (lib/)

```
lib/
|-- main.dart
|-- firebase_options.dart
|-- models/                (12 archivos)
|-- services/              (9 archivos)
|-- screens/               (19 pantallas)
    |-- reciclaje/         (28 componentes UI reutilizables)
```

El proyecto sigue una arquitectura por capas simple: **models** (clases de datos con fromMap/toMap para Firestore), **services** (acceso a Firestore/Auth, sin lógica de UI) y **screens** (widgets de pantalla, con screens/reciclaje/ como biblioteca de componentes visuales compartidos: botones, cards, badges, tema, etc.). No hay carpeta de *providers*, *controllers* ni gestor de estado externo: cada pantalla maneja su propio estado con StatefulWidget + StreamBuilder/FutureBuilder directos sobre los servicios.

1.2 lib/models — Modelos de datos

Archivo	Qué hace
admin_model.dart	Administrador autorizado (colección admins).
auditoria_model.dart	Registro de auditoría; el modelo existe pero nada en el código escribe en la colección 'auditoria' (ver Sección 7).
campeonato_model.dart	Modelo central: CampeonatoModel, CampeonatoConfig, ReglasPuntuacion y los enums de deporte/formato/tipo de resultado.
equipo_model.dart	EquipoModel + estados de equipo (activo/inactivo/retirado/descalificado).
gol_model.dart	Un gol de un jugador en un partido (colección 'goles', solo fútbol/futsal).
grupo_model.dart	Modelo de grupo de fase de grupos. Definido pero no usado por ningún servicio ni pantalla (ver Sección 7).
historial_cambio_model.dart	Registro de auditoría de ediciones de jugador/equipo (sí se usa).
jugador_model.dart	JugadorModel + estados de jugador (activo/suspendido/retirado).
model_helpers.dart	Funciones puras de parseo seguro Firestore->Dart (stringFromJson, intFromJson, boolFromJson, dateFromJson/dateToJson, stringListFromJson). Usadas por todos los demás modelos.
partido_model.dart	PartidoModel + estados de partido, tipos de resultado/definición y SetPartido (detalle de sets de vóley).
ranking_goleador_model.dart	Fila del ranking de goleadores (colección 'ranking_goleadores').
tabla_posicion_model.dart	Fila de la tabla de posiciones de un equipo (colección 'tabla_posiciones').

1.3 lib/services — Acceso a datos

Archivo	Qué hace
admin_service.dart	CRUD de administradores sobre la colección 'admins'.
auth_service.dart	Login/logout con Firebase Auth; verifica que el usuario autenticado tenga un documento activo en 'admins' antes de dejarlo pasar.

Archivo	Qué hace
campeonato_service.dart	CRUD de campeonatos y transición de estados (inscripción -> activo -> finalizado), incluida la validación previa a activar.
equipo_service.dart	CRUD de equipos; al crear un equipo también crea su fila inicial en 'tabla_posiciones'.
jugador_service.dart	Alta/edición de jugadores, con transacción que valida cupo máximo por equipo y guarda historial de cambios.
partido_service.dart	Generador de fixture (round-robin, fase de grupos, eliminación directa), cruces manuales y programación/reprogramación de partidos.
pdf_service.dart	Generación de todos los documentos PDF de la app (fixture, resultados, planillas, listas de equipo, ranking).
public_home_service.dart	Streams de solo lectura para la vista pública (home y detalle público), sin requerir sesión.
resultado_service.dart	Registro de resultados (goles/sets/puntos según deporte) y recálculo de la tabla de posiciones y el ranking de goleadores.

1.4 lib/screens — Pantallas

Ver el detalle completo de cada pantalla en la Sección 4. Listado de archivos:

- admin_panel_screen.dart
- auditoria_screen.dart
- campeonato_form_screen.dart
- campeonatos_screen.dart
- championship_detail_screen.dart
- detalle_campeonato_screen.dart
- equipo_form_screen.dart
- equipos_screen.dart
- fixture_screen.dart
- historial_jugador_screen.dart
- home_screen.dart
- jugador_form_screen.dart
- jugadores_screen.dart
- login_screen.dart
- pdfs_screen.dart
- ranking_goleadores_screen.dart
- resultado_form_screen.dart
- resultados_screen.dart
- tabla_posiciones_screen.dart

1.5 lib/screens/reciclaje — Sistema de diseño / componentes UI

Archivo(s)	Qué hace
app_theme.dart	ThemeData de Material 3 completo: ColorScheme, TextTheme, temas de botón/diálogo/snackbar/etc.
app_colors.dart	Paleta de colores institucional UPSA (verde primario, dorado secundario).
app_text_styles.dart	Escala tipográfica (Google Fonts 'Inter').
app_sizes.dart	Constantes de espaciado y radios de borde.

Archivo(s)	Qué hace
responsive.dart	Helpers de breakpoints (móvil/tablet/desktop).
app_page.dart	Layout de página estándar: título, subtítulo, acciones, contenido con ancho máximo.
app_button.dart	Botón con 4 variantes (primary/secondary/danger/ghost) sobre FilledButton/OutlinedButton/TextButton.
app_card.dart	Card institucional con sombra y esquinas redondeadas.
app_badge.dart	Etiqueta de estado con color según tipo.
app_text_field.dart	Campo de texto con label, ícono y validación integrados.
app_dialogs.dart	Diálogos de confirmación y de solicitud de observación.
app_snackbars.dart	Snackbars de éxito/error/advertencia/información.
app_loading.dart	Indicador de carga con fade-in.
app_skeleton.dart	Placeholders tipo 'shimmer' para listas en carga.
app_empty_state.dart / app_inline_empty_state.dart	Estados vacíos (pantalla completa / en línea).
app_hero_card.dart	Banner superior con degradado institucional, usado en casi todas las pantallas.
app_match_card.dart	Tarjeta de partido (marcador, equipos, fecha).
app_standing_card.dart	Tarjeta de posición de equipo en tabla (vista móvil).
championship_public_card.dart	Tarjeta de campeonato en el listado público.
stat_card.dart	Tarjeta de estadística numérica (KPI).
app_table_container.dart	Tabla de datos con scroll horizontal.
app_section_header.dart / app_info_line.dart / app_logo_mark.dart / app_filter_pill.dart / app_responsive_grid.dart / app_responsive_pair.dart	Componentes auxiliares de layout y filtros.

Sección 2 — Modelos de datos

Todos los modelos siguen el mismo patrón: campos `final`, constructor inmutable, `factory fromMap(id, map)` para leer de Firestore (con valores por defecto seguros vía `model_helpers.dart`) y `toMap()` para escribir. A continuación el detalle completo de cada clase.

AdminModel (admin_model.dart)

Campo	Tipo
id	String
nombre	String
email	String
activo	bool
fechaCreacion	DateTime?
ultimaConexion	DateTime?

Métodos / getters relevantes:

- `copyWith(...)` para clonar con cambios parciales.

AuditoriaModel (auditoria_model.dart)

Campo	Tipo
id	String
usuarioId	String
usuarioNombre	String
accion	String
modulo	String
documentoAfectado	String
fecha	DateTime?
detalle	String
observacion	String?

CampeonatoModel (campeonato_model.dart)

Campo	Tipo
id	String
nombre	String
descripcion	String
deporte	String
modalidad	String
tipoCampeonato	String
estado	String

Campo	Tipo
temporada	String
cancha	String
configuracion	CampeonatoConfig
reglasPuntuacion	ReglasPuntuacion
reglasDesempate	List
fechaCreacion	DateTime?
fechaActualizacion	DateTime?
creadoPor	String

Métodos / getters relevantes:

- estaEnInscripcion / estaActivo / estaFinalizado — getters de estado.
- deporteEfectivo — devuelve 'futbol' si el campo viene vacío (compatibilidad con campeonatos antiguos).
- esFutbol / esVolley / esBasket — getters por deporte.
- sistemaResultadoEfectivo — deduce goles/sets/puntos según el deporte si el campo no está seteado.
- formatoEliminaEmpates — true solo para tipoCampeonato == eliminacion_directa.

CampeonatoConfig (clase anidada en campeonato_model.dart) (campeonato_model.dart)

Campo	Tipo
formato	String
cantidadVueltas	int
idaYVuelta	bool
generaCrucesAleatorios	bool
generaGruposAleatorios	bool
permiteEmpate	bool
generaTablaPosiciones	bool
cantidadJugadoresEnCancha	int
cantidadMinimaJugadoresPorEquipo	int
cantidadMaximaJugadoresPorEquipo	int
cantidadGrupos	int
clasificanPorGrupo	int
clasificadosPlayoffs	int
idaYVueltaEnGrupos	bool
incluyeTercerLugar	bool
generaFaseEliminatoria	bool
fixtureManualPermitido	bool
rondaEliminatoriaInicial	String
deporte	String
modalidad	String

Campo	Tipo
sistemaResultado	String
setsParaGanar	int
puntosSetNormal	int
puntosSetDecisivo	int
cantidadPeriodos	int
permitePenales	bool
permiteProrroga	bool

ReglasPuntuacion (clase anidada) (campeonato_model.dart)

Campo	Tipo
victoria	int (default 3)
empate	int (default 1)
derrota	int (default 0)

EquipoModel (equipo_model.dart)

Campo	Tipo
id	String
nombre	String
representante	String
carrera	String?
facultad	String?
estado	String
cantidadJugadoresRegistrados	int
fechaCreacion	DateTime?
fechaActualizacion	DateTime?
creadoPor	String
actualizadoPor	String

GolModel (gol_model.dart)

Campo	Tipo
id	String
partidoId	String
equipoId	String
equipoNombre	String
jugadorId	String

Campo	Tipo
jugadorNombre	String
cantidad	int
fechaRegistro	DateTime?
registradoPor	String

GrupoModel [!] no usado en el código actual (grupo_model.dart)

Campo	Tipo
id	String
nombre	String
orden	int
equipolds	List
generadoPorSistema	bool
fechaCreacion	DateTime?

HistorialCambioModel (historial_cambio_model.dart)

Campo	Tipo
id	String
accion	String
datosAnteriores	Map
datosNuevos	Map
observacion	String
usuariold	String
usuarioNombre	String
fecha	DateTime?

JugadorModel (jugador_model.dart)

Campo	Tipo
id	String
equipold	String
equipoNombre	String
codigoEstudiante	String
nombreCompleto	String
estado	String
fechaRegistro	DateTime?
fechaActualizacion	DateTime?

Campo	Tipo
creadoPor	String
actualizadoPor	String
ultimaObservacion	String?

Métodos / getters relevantes:

- estaActivo / estaSuspendido / estaRetirado — getters de estado.

PartidoModel (partido_model.dart)

Campo	Tipo
id	String
jornada	int
vuelta	int
grupold	String?
equipoLocalId	String
equipoLocalNombre	String
equipoVisitanteId	String
equipoVisitanteNombre	String
fechaHora	DateTime?
estado	String
golesLocal	int?
golesVisitante	int?
ganadorId	String?
empate	bool
resultadoRegistrado	bool
generadoPorSistema	bool
tipoResultado	String
observacionResultado	String?
fechaCreacion	DateTime?
fechaActualizacion	DateTime?
penalesLocal	int?
penalesVisitante	int?
definidoPorPenales	bool
definidoPorProrroga	bool
tipoDefinicion	String
sets	List

Métodos / getters relevantes:

- estaPendienteProgramacion / estaProgramado / estaFinalizado / estaSuspendido — getters de estado.

- marcadorTexto — arma el marcador legible, p.ej. '1 - 1 (4 - 3 pen.)' o '(prórroga)'.
- setsTexto — arma el detalle de sets, p.ej. '25-20 · 23-25 · 15-12'.

SetPartido (clase anidada en partido_model.dart) (partido_model.dart)

Campo	Tipo
local	int
visitante	int

RankingGoleadorModel (ranking_goleador_model.dart)

Campo	Tipo
jugadorId	String
jugadorNombre	String
jugadorEstado	String
equipold	String
equipoNombre	String
totalGoles	int
partidosConGol	int
fechaActualizacion	DateTime?

TablaPosicionModel (tabla_posicion_model.dart)

Campo	Tipo
equipold	String
equipoNombre	String
partidosJugados	int
partidosGanados	int
partidosEmpatados	int
partidosPerdidos	int
golesFavor	int
golesContra	int
diferenciaGoles	int
puntos	int
posicion	int
fechaActualizacion	DateTime?
puntosFavor	int
puntosContra	int
diferenciaPuntos	int

Métodos / getters relevantes:

- factory `TablaPosicionModel.empty(...)` — fila inicial en cero para un equipo recién creado.
- En vóley, `golesFavor/golesContra` almacenan SETS a favor/en contra (no goles); `puntosFavor/puntosContra` almacenan los puntos jugados dentro de los sets.

Sección 3 — Base de datos / Backend

3.1 Tecnología

- **Firebase Auth** (paquete `firebase_auth: ^6.5.2`) — login por email/contraseña.
- **Cloud Firestore** (paquete `cloud_firestore: ^6.5.0`) — base de datos NoSQL en tiempo real; toda la app lee/escribe con `StreamBuilder`/consultas directas, sin caché ni repositorio intermedio.
- **firebase_core: ^4.10.0** — inicialización de la app Firebase.
- No hay backend propio (sin API REST, sin servidor Node/Express, sin Cloud Functions detectadas en el código Flutter). Toda la lógica de negocio corre en el cliente Flutter.
- No se usa SQLite, Hive, ni ninguna base de datos local: no hay soporte offline explícito más allá del que Firestore da por defecto.

3.2 Conexión / credenciales

La app se conecta mediante `lib/firebase_options.dart`, generado por el FlutterFire CLI. Este archivo contiene las claves de configuración (`apiKey`, `appId`, `projectId`, etc.) **embebidas directamente en el código fuente** para las plataformas web, Android, iOS, macOS y Windows — no se usa un archivo `.env` ni variables de entorno. El proyecto de Firebase es **futsal-37be1**.

Nota de seguridad

Las claves de Firebase están hardcodeadas en `firebase_options.dart` y por lo tanto se suben al control de versiones. Esto es el comportamiento estándar de FlutterFire (la `apiKey` de Firebase para apps cliente no es secreta por diseño; la protección real depende de las reglas de seguridad de Firestore/Auth, que no forman parte de este repositorio Flutter y no pudieron ser revisadas aquí).

`main.dart` inicializa Firebase con `Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform)` antes de correr `MyApp`.

3.3 Colecciones de Firestore

Estructura observada (todas anidadas bajo `campeonatos/{campeonatoId}` salvo 'admins'):

Colección	Nivel	Quién la usa	Campos principales
admins	raíz	id = uid de Firebase Auth	nombre, email, activo, fechaCreacion, ultimaConexion
campeonatos	raíz	AdminService/CampeonatoService	nombre, descripcion, deporte, modalidad, tipoCampeonato, estado, temporada, cancha, configuracion {...}, reglasPuntuacion {...}, reglasDesempate[], fechaCreacion, fechaActualizacion, creadoPor
campeonatos/{id}/equipos	sub-colección	EquipoService	nombre, representante, carrera, facultad, estado, cantidadJugadoresRegistrados, fechaCreacion, fechaActualizacion, creadoPor, actualizadoPor
.../equipos/{id}/historial_planilla	sub-sub-colección	EquipoService.editarEquipo	accion, datosAnteriores, datosNuevos, observacion, usuarioId, usuarioNombre, fecha
campeonatos/{id}/jugadores	sub-colección	JugadorService (id = código de estudiante normalizado)	equipoId, equipoNombre, codigoEstudiante, codigoNormalizado, nombreCompleto, estado, fechaRegistro, fechaActualizacion, creadoPor, actualizadoPor, ultimaObservacion
.../jugadores/{id}/historial_cambios	sub-sub-colección	JugadorService.editarJugador	accion, datosAnteriores, datosNuevos, observacion, usuarioId, usuarioNombre, fecha

Colección	Nivel	Quién la usa	Campos principales
campeonatos/{id}/partidos	sub-colección	PartidoService/ResultadoService	jornada, vuelta, grupalId, equipoLocalId/Nombre, equipoVisitanteId/Nombre, fechaHora, estado, golesLocal, golesVisitante, ganadorId, empate, resultadoRegistrado, generadoPorSistema, tipoResultado, observacionResultado, penalesLocal, penalesVisitante, definidoPorPenales, definidoPorProrroga, tipoDefinicion, sets[], fechaCreacion, fechaActualizacion
campeonatos/{id}/goles	sub-colección	ResultadoService (solo fútbol/futsal)	partidoId, equipoid, equipoNombre, jugadorId, jugadorNombre, cantidad, fechaRegistro, registradoPor
campeonatos/{id}/tarjetas	sub-colección	ResultadoService (solo fútbol/futsal)	partidoId, equipoid, equipoNombre, jugadorId, jugadorNombre, amarillas, rojas, motivo, fechaRegistro, registradoPor
campeonatos/{id}/tabla_posiciones	sub-colección	EquipoService (fila inicial) + ResultadoService (recálculo)	ver TablaPosicionModel en Sección 2
campeonatos/{id}/ranking_goleadores	sub-colección	ResultadoService (recálculo)	ver RankingGoleadorModel en Sección 2
campeonatos/{id}/auditoria	sub-colección	solo LECTURA (auditoria_screen.dart) — nada la escribe	usuarioid, usuarioNombre, accion, modulo, documentoAfectado, fecha, detalle, observacion

Nota: 'goles' y 'tarjetas' se consultan directamente con `Firestore.instance` desde las pantallas (`tabla_posiciones_screen.dart`, `pdfs_screen.dart`) en vez de pasar por un service dedicado — es la única inconsistencia arquitectónica de acceso a datos detectada (ver Sección 7).

Sección 4 — Pantallas y navegación

La navegación es 100% imperativa: no hay named routes ni paquete de routing (go_router, etc.). Cada pantalla se abre con `Navigator.push(context, MaterialPageRoute(builder: (_) => Pantalla(...)))`. `main.dart` define un único punto de entrada: `MaterialApp(home: HomeScreen())`.

4.1 Mapa de navegación

HomeScreen (publica, sin login)

- |-> LoginScreen -> AdminPanelScreen
- `-> ChampionshipDetailScreen (detalle publico de un campeonato)

AdminPanelScreen (requiere admin activo)

- |-> CampeonatosScreen
- |-> HomeScreen (ver vista publica)
- `-> ChampionshipDetailScreen

CampeonatosScreen

- |-> CampeonatoFormScreen (nuevo campeonato)
- `-> DetalleCampeonatoScreen (al tocar un campeonato)

DetalleCampeonatoScreen (hub administrativo, 8 modulos)

- |-> EquiposScreen -> EquipoFormScreen
- |-> JugadoresScreen -> JugadorFormScreen
- |-> FixtureScreen (+ dialogo de cruce manual)
- |-> ResultadosScreen -> ResultadoFormScreen
- |-> TablaPosicionesScreen (en realidad muestra HISTORIAL de partidos, ver mas abajo)
- |-> RankingGoleadoresScreen
- |-> PdfsScreen
- `-> AuditoriaScreen (siempre vacia, ver Seccion 7)

HistorialJugadorScreen -- existe pero NINGUNA pantalla navega hacia ella (huerfana).

4.2 Detalle por pantalla

HomeScreen — Portal público

`home_screen.dart` — widget principal: `HomeScreen (StatefulWidget)`

Cómo se llega: Pantalla inicial de la app (home de `MaterialApp`). Acceso público, sin login.

Qué muestra: Hero institucional, buscador y filtros (todos/inscripción/activo/finalizado), y una grilla responsive de `ChampionshipPublicCard` con todos los campeonatos (stream en tiempo real de `PublicHomeService`).

Acciones disponibles:

- Buscar por nombre/temporada/modalidad/cancha.
- Filtrar por estado del campeonato.
- Tocar un campeonato -> `ChampionshipDetailScreen`.
- Botón 'Ingresar admin' -> `LoginScreen`.

LoginScreen — Ingreso de administrador

`login_screen.dart` — widget principal: `LoginScreen (StatefulWidget)`

Cómo se llega: Desde el botón 'Ingresar admin' de `HomeScreen`.

Qué muestra: Formulario de correo y contraseña dentro de una card centrada.

Acciones disponibles:

- Iniciar sesión (AuthService.login) -> si es admin activo, pasa a AdminPanelScreen.
- Mostrar/ocultar contraseña.
- Volver al inicio.

AdminPanelScreen — Panel administrativo

admin_panel_screen.dart — widget principal: AdminPanelScreen (StatefulWidget)

Cómo se llega: Tras un login exitoso (Navigator.pushAndRemoveUntil, reemplaza el stack).

Qué muestra: Hero de bienvenida con nombre del admin, estadísticas (total/activos/próximamente/finalizados) y listado resumido de campeonatos registrados.

Acciones disponibles:

- Gestionar campeonatos -> CampeonatosScreen.
- Ver pantalla pública -> HomeScreen.
- Cerrar sesión (logout + vuelve a HomeScreen).
- Por cada campeonato: 'Gestionar' -> CampeonatosScreen, 'Ver público' -> ChampionshipDetailScreen.

CampeonatosScreen — Listado administrativo de campeonatos

campeonatos_screen.dart — widget principal: CampeonatosScreen (StatefulWidget)

Cómo se llega: Desde AdminPanelScreen ('Gestionar campeonatos').

Qué muestra: Hero, estadísticas, buscador/filtros y grilla de campeonatos en formato administrativo (con botón 'Administrar campeonato').

Acciones disponibles:

- Crear nuevo campeonato -> CampeonatoFormScreen.
- Buscar y filtrar por estado.
- Tocar un campeonato -> DetalleCampeonatoScreen.

CampeonatoFormScreen — Alta de campeonato

campeonato_form_screen.dart — widget principal: CampeonatoFormScreen (StatefulWidget)

Cómo se llega: Desde CampeonatosScreen ('Nuevo campeonato').

Qué muestra: Formulario largo: datos generales, selección de deporte (fútbol/vóley/básquet) y modalidad, configuración deportiva dinámica (jugadores en cancha, mín/máx por equipo, sets/periodos), selector visual de formato de campeonato (7 formatos) y reglas dinámicas según el formato elegido.

Acciones disponibles:

- Guardar (crea el campeonato en estado 'inscripción').
- Cancelar / volver.
- Validaciones en vivo (autovalidateMode) de mínimos/máximos y reglas por deporte.

DetalleCampeonatoScreen — Hub administrativo del campeonato

detalle_campeonato_screen.dart — widget principal: DetalleCampeonatoScreen (StatefulWidget)

Cómo se llega: Desde CampeonatosScreen al tocar un campeonato.

Qué muestra: Hero con nombre/estado/modalidad, estadísticas de configuración y una grilla de 8 módulos (Equipos, Jugadores, Fixture, Resultados, Tabla, Goleadores/Anotadores, PDFs, Auditoría) más un resumen textual de la configuración guardada.

Acciones disponibles:

- Activar campeonato (valida equipos/jugadores mínimos).
- Finalizar campeonato.
- Navegar a cualquiera de los 8 módulos.

EquiposScreen / EquipoFormScreen — Equipos

`equipos_screen.dart`, `equipo_form_screen.dart` — widget principal: `EquiposScreen (Stateless)`, `EquipoFormScreen (Stateful)`

Cómo se llega: Desde el módulo 'Equipos' de `DetalleCampeonatoScreen`.

Qué muestra: Grilla de equipos (nombre, representante, carrera, facultad, jugadores registrados, estado). El formulario permite crear o editar un equipo.

Acciones disponibles:

- Crear equipo.
- Editar equipo (si el campeonato está activo, exige una observación obligatoria).

JugadoresScreen / JugadorFormScreen — Jugadores

`jugadores_screen.dart`, `jugador_form_screen.dart` — widget principal: `JugadoresScreen (Stateless)`, `JugadorFormScreen (Stateful)`

Cómo se llega: Desde el módulo 'Jugadores' de `DetalleCampeonatoScreen`.

Qué muestra: Tabla de jugadores (código, nombre, equipo, estado). El formulario permite alta individual o registro masivo por tabla de filas (`_JugadoresMasivosForm`) y edición de un jugador existente.

Acciones disponibles:

- Registrar jugador (valida cupo máximo por equipo vía transacción).
- Editar jugador (exige observación si el campeonato está activo).
- Registro masivo de varios jugadores a la vez.

FixtureScreen — Fixture y programación

`fixture_screen.dart` — widget principal: `FixtureScreen (StatefulWidget)`

Cómo se llega: Desde el módulo 'Fixture' de `DetalleCampeonatoScreen`.

Qué muestra: Estadísticas del fixture (total/programados/pendientes/manuales), filtros, y partidos agrupados por grupo/vuelta/ronda con tarjetas de partido responsive (`AppMatchCard`).

Acciones disponibles:

- Generar fixture automático según el formato del campeonato (round-robin, grupos o eliminación).
- Agregar cruce manual (diálogo con selección de equipos, jornada, grupo/fase e ida-vuelta).
- Programar / reprogramar fecha y hora de un partido.
- Descargar el fixture en PDF.

ResultadosScreen / ResultadoFormScreen — Resultados

`resultados_screen.dart`, `resultado_form_screen.dart` — widget principal: `ResultadosScreen (Stateless)`, `ResultadoFormScreen (Stateful)`

Cómo se llega: Desde el módulo 'Resultados' de `DetalleCampeonatoScreen`.

Qué muestra: Listado de partidos con su marcador (o 'Sin resultado'), badges de grupo/penales/prórroga. El formulario cambia según el deporte: goles + goles por jugador + tarjetas (fútbol), sets (vóley), puntos + posible prórroga (básquet); penales cuando el formato no permite empate.

Acciones disponibles:

- Registrar/editar resultado (solo si el campeonato está activo y el partido no está pendiente de programación).
- Marcar resultado como walkover o sanción.

TablaPosicionesScreen — En realidad: historial de partidos

`tabla_posiciones_screen.dart` — widget principal: `TablaPosicionesScreen` (`StatefulWidget`)

Cómo se llega: Desde el módulo 'Tabla' de `DetalleCampeonatoScreen`.

Qué muestra: [!] A pesar del nombre y del ítem del menú, esta pantalla NO muestra la tabla de posiciones: muestra el HISTORIAL de partidos jugados, con goles y tarjetas por jugador de cada partido. La tabla de posiciones real solo se ve en `ChampionshipDetailScreen` (vista pública).

Acciones disponibles:

- Ver detalle de cada partido finalizado (goleadores, tarjetas, observación).
- Actualizar.

RankingGoleadoresScreen — Ranking de goleadores

`ranking_goleadores_screen.dart` — widget principal: `RankingGoleadoresScreen` (`StatelessWidget`)

Cómo se llega: Desde el módulo 'Goleadores'/'Anotadores' de `DetalleCampeonatoScreen`.

Qué muestra: Para fútbol/futsal: lista ordenada de goleadores con posición, goles y estado del jugador. Para vóley/básquet: mensaje de 'módulo en preparación' (no hay estadística individual todavía).

Acciones disponibles:

- Solo lectura (sin acciones de edición).

PdfsScreen — Centro de impresión

`pdfs_screen.dart` — widget principal: `PdfsScreen` (`StatefulWidget`)

Cómo se llega: Desde el módulo 'PDFs' de `DetalleCampeonatoScreen`.

Qué muestra: Panel con selectores de rango de fechas y botones para cada tipo de documento imprimible (ver Sección 6.4 para el detalle completo de los 7 PDFs).

Acciones disponibles:

- Generar y abrir en el visor de impresión: fixture completo, fixture por rango, resultados por rango, ranking de goleadores, planilla de un partido, lista de un equipo, listas de todos los equipos.

AuditoriaScreen — Auditoría (siempre vacía)

`auditoria_screen.dart` — widget principal: `AuditoriaScreen` (`StatelessWidget`)

Cómo se llega: Desde el módulo 'Auditoría' de `DetalleCampeonatoScreen`.

Qué muestra: Lista de registros de auditoría leídos de `campeonatos/{id}/auditoria`.

Acciones disponibles:

- Solo lectura. En la práctica siempre se ve el estado vacío porque nada en el código escribe en esa colección (ver Sección 7).

HistorialJugadorScreen — Pantalla huérfana

`historial_jugador_screen.dart` — widget principal: `HistorialJugadorScreen` (`StatelessWidget`)

Cómo se llega: [!] Ninguna otra pantalla la referencia (no hay ningún `Navigator.push` hacia ella en todo el proyecto). El historial de cambios de un jugador SÍ existe y se guarda (`historial_cambios` en `jugador_service.dart`), pero no hay forma de llegar a esta pantalla desde la interfaz.

Qué muestra: Lista de `HistorialCambioModel` de un jugador puntual, con observación, datos anteriores y nuevos.

Acciones disponibles:

- Solo lectura (código funcional, pero inalcanzable desde la UI actual).

ChampionshipDetailScreen — Detalle público de campeonato

`championship_detail_screen.dart` — widget principal: `ChampionshipDetailScreen` (`StatelessWidget`)

Cómo se llega: Desde `HomeScreen` o `AdminPanelScreen` ('Ver público'), sin necesidad de login.

Qué muestra: Vista 100% pública: hero, estadísticas, próximos partidos, últimos resultados, tabla de posiciones REAL (adaptada por deporte: fútbol/vóley/básquet) y goleadores.

Acciones disponibles:

- Solo lectura — es el 'sitio web' público del campeonato.
- Volver.

Sección 5 — Sistema de roles y autenticación

5.1 Cómo se maneja el login

El login (`AuthService.login` en `auth_service.dart`) usa `FirebaseAuth.signInWithEmailAndPassword`. Después de autenticar, verifica que exista un documento con ese UID en la colección `admins` y que su campo `activo` sea `true`; si no, cierra la sesión inmediatamente y lanza una excepción ('Esta cuenta no tiene permisos de administrador'). Si el login es válido, actualiza `ultimaConexion` con la hora del servidor.

- `authChanges` — stream de `FirebaseAuth.authStateChanges()`.
- `esAdminActivo(uid)` — `true` si el doc en 'admins' existe y `activo==true`.
- `getAdminActual()` / `adminActualStream()` — traen el `AdminModel` del usuario logueado.
- `requireAdmin()` — usado por casi todos los formularios antes de guardar; lanza excepción si no hay admin activo.

5.2 Roles existentes

El sistema tiene **un único rol real: 'administrador'**. No existen roles intermedios (editor, moderador, capitán de equipo, etc.). La distinción binaria es:

- **Usuario público / visitante:** no necesita cuenta. Accede solo a `HomeScreen` y `ChampionshipDetailScreen` (información de solo lectura vía `PublicHomeService`).
- **Administrador:** cuenta de Firebase Auth con un documento correspondiente y activo en la colección 'admins'. Tiene acceso a `AdminPanelScreen` y a todo el árbol de gestión (campeonatos, equipos, jugadores, fixture, resultados, PDFs).

No hay roles intermedios

No existe, por ejemplo, un rol de 'capitán de equipo' que solo pueda ver/editar su propio equipo, ni un rol de 'árbitro' limitado a cargar resultados. Todo admin activo tiene acceso total a todos los campeonatos.

5.3 Cómo se diferencia un admin de un usuario normal en el código

No hay *custom claims* de Firebase Auth ni roles embebidos en el token: la verificación es siempre una consulta a Firestore. El patrón repetido en cada pantalla/servicio administrativo es:

```
final admin = await _authService.requireAdmin();  
// ... si no hay admin activo, lanza Exception y el catch muestra un snackbar de error
```

Las pantallas públicas (`HomeScreen`, `ChampionshipDetailScreen`) usan `PublicHomeService`, que NO valida sesión alguna: cualquier persona con la URL de la app puede leer campeonatos, equipos, partidos, tabla y goleadores. La protección real de escritura depende de las reglas de seguridad de Firestore, que no están en este repositorio Flutter (no se pudieron revisar).

5.4 Qué puede hacer cada rol

Rol	Qué puede hacer
Visitante (sin login)	Ver <code>HomeScreen</code> , buscar/filtrar campeonatos, ver <code>ChampionshipDetailScreen</code> (próximos partidos, últimos resultados, tabla de posiciones, goleadores). No puede crear, editar ni eliminar nada.
Administrador	Todo lo del visitante, más: crear/activar/finalizar campeonatos; crear/editar equipos y jugadores (con observación obligatoria si el campeonato está activo); generar fixture automático o manual; programar partidos; registrar resultados; generar todos los PDFs; ver el módulo de auditoría (aunque siempre está vacío, ver Sección 7).

Sección 6 — Lógica deportiva actual

6.1 Tabla de posiciones — cálculo y orden

Implementada en `ResultadoService._recalcularTabla` (`resultado_service.dart`). Se recalcula **completa** cada vez que se guarda un resultado (no es incremental): borra todas las filas existentes de 'tabla_posiciones' y las vuelve a generar desde cero a partir de todos los partidos con estado `== finalizado` y `resultadoRegistrado == true`.

- **Puntos:** usa `CampeonatoModel.reglasPuntuacion` (victoria=3, empate=1, derrota=0 por defecto), igual para los tres deportes.
- **Goles/sets a favor y en contra:** si el partido tiene detalle de sets (vóley), se suman los puntos de cada set; si no, se usa directamente `golesLocal/golesVisitante` (fútbol y básquet, donde 'goles' son en realidad los puntos del marcador).
- **Orden de la tabla (fijo, igual para los tres deportes):** puntos (desc) -> diferencia de goles/sets (desc) -> goles/sets a favor (desc) -> goles/sets en contra (asc) -> nombre del equipo (alfabético).

El campo `reglasDesempate` no se usa en el cálculo real

Al crear un campeonato, `CampeonatoService` guarda una lista de `reglasDesempate` distinta por deporte (p.ej. para vóley: puntos, diferencia_sets, sets_favor, diferencia_puntos, resultado_directo). Sin embargo, el ordenamiento real en `_recalcularTabla` no lee ese campo en ningún momento: siempre aplica el mismo criterio fijo descrito arriba. El campo se guarda pero es efectivamente decorativo.

6.2 Fases: grupos, eliminatorias y brackets

`CampeonatoConfig.tipoCampeonato` soporta 7 formatos (`TipoCampeonato`):

- `solo_ida`, `ida_vuelta` — liga todos-contra-todos, 1 o 2 vueltas.
- `liga_final`, `liga_playoffs` — liga + fase final.
- `fase_grupos`, `grupos_eliminacion` — grupos + (opcional) fase eliminatoria.
- `eliminacion_directa` — llaves de partido único desde la primera ronda.

El generador automático (`PartidoService.generarFixtureSegunFormato`) cubre:

- **Round-robin** (`solo_ida/ida_vuelta` y también la fase de liga de `liga_final/liga_playoffs`): algoritmo de rotación clásico, con alternancia de localía.
- **Fase de grupos:** reparte los equipos en 'Grupo A', 'Grupo B', etc. de forma aleatoria o 'tipo serpiente' (distribución pareja), y genera round-robin dentro de cada grupo. El identificador de grupo es un `String` `grupoId` guardado en cada partido — **no** se crea un documento de grupo aparte.
- **Eliminación directa:** cruces aleatorios de la primera ronda; si la cantidad de equipos activos es impar, lanza una excepción explicando que se debe ajustar la cantidad de equipos o usar un cruce manual.

Las fases finales no se generan automáticamente

Para `liga_final`, `liga_playoffs` y `grupos_eliminacion`, el generador crea solo la fase inicial (liga o grupos). La fase final/eliminatoria posterior (semifinales, final, tercer puesto, o las rondas siguientes de una eliminación directa) **no se arma sola**: el propio código de `FixtureScreen` se lo explica al usuario y lo dirige a usar 'Agregar cruce manual' una vez que la tabla o los clasificados estén definidos. Es una decisión deliberada del código (no inventar clasificados sin resultados reales), pero implica trabajo manual del administrador en esos 3 formatos.

6.3 Registro de partidos y resultados

PartidoService crea los partidos en estado pendiente_programacion; desde FixtureScreen se pueden programar/reprogramar fecha y hora, y agregar cruces manuales aunque el fixture ya exista (mientras el campeonato no esté finalizado).

ResultadoService.registrarResultado ramifica su validación según campeonato.sistemaResultadoEfectivo:

- **Fútbol/futsal (goles):** valida que la suma de goles cargados por jugador coincida con el marcador final. Si el marcador queda empatado y el formato no permite empate (!permiteEmpate o tipoCampeonato == eliminacion_directa), exige penalesLocal/penalesVisitante distintos y define el ganador por penales.
- **Vóley (sets):** exige el detalle set por set, valida que ningún set termine empatado, que el ganador alcance setsParaGanar y que no ganen ambos equipos.
- **Básquet (puntos):** no admite empate; si el marcador queda igualado exige marcar definidoPorProrroga.
- **walkover / sanción:** resultado administrativo (p.ej. 3-0), sin goles/tarjetas por jugador, con observación obligatoria.

Al final de cualquier registro exitoso se dispara siempre recalcularTablaYRanking() (tabla + ranking de goleadores completos).

6.4 PDFs que genera la app

Método (pdf_service.dart)	Se dispara desde	Contenido
generarFixturePdf	PdfsScreen -> 'Imprimir fixture completo'	Programación completa del campeonato.
generarFixturePorRangoPdf	PdfsScreen -> sección Fixture, con selector de fecha inicio/fin	Partidos programados dentro de un rango de fechas.
generarResultadosPorRangoPdf	PdfsScreen -> sección Resultados, con selector de fecha inicio/fin	Resultados finalizados en un rango, incluyendo goleadores por partido (consulta 'goles' directo).
generarRankingGoleadoresPdf	PdfsScreen -> 'Ranking de goleadores'	Top 10 goleadores (consulta 'ranking_goleadores' directo).
generarPlanillaPartidoPdf	PdfsScreen -> sección de planillas, por cada partido	Planilla de control de un partido puntual, con nómina de ambos equipos.
generarListaEquipoPdf	PdfsScreen -> sección de equipos, por cada equipo	Nómina de un equipo.
generarListasEquiposPdf	PdfsScreen -> 'Imprimir todas las listas'	Nóminas de todos los equipos en un solo PDF.

Todos abren el diálogo de impresión/descarga del navegador con Printing.layoutPdf. Los títulos usan el deporte real del campeonato (FÚTBOL/VÓLEY/BÁSQUET) en vez de tener 'FÚTBOL' fijo, pero el diseño interno de la planilla (columnas de goles/tarjetas) es el mismo para los tres deportes — ver Sección 7.

Sección 7 — Estado actual y lo que falta

7.1 Funcionalidades completas

- Autenticación de administrador con verificación de rol contra Firestore (no custom claims).
- CRUD completo de campeonatos, equipos y jugadores, con historial de auditoría de ediciones (historial_planilla / historial_cambios) cuando el campeonato está activo.
- Selección de deporte (fútbol/vóley/básquet) y modalidad, con configuración deportiva dinámica (jugadores en cancha, mínimos/máximos, sets, periodos) en el alta de campeonato.
- Generación automática de fixture para liga (solo ida / ida y vuelta), fase de grupos y primera ronda de eliminación directa, más cruces manuales en cualquier momento.
- Programación y reprogramación de partidos.
- Registro de resultados diferenciado por deporte (goles/sets/puntos), con penales en fútbol y prórroga en básquet cuando el formato no admite empate.
- Cálculo y visualización en tiempo real de tabla de posiciones (adaptada por deporte) y ranking de goleadores (fútbol).
- 7 tipos de documentos PDF descargables/imprimibles.
- Vista pública sin necesidad de login (home + detalle de campeonato).
- UI responsive (móvil/tablet/desktop) con un sistema de diseño Material 3 propio (lib/screens/reciclaje).

7.2 Funcionalidades a medias (documentadas como tales en el propio código)

- **Fases finales de liga_final / liga_playoffs / grupos_eliminacion:** solo se genera la fase inicial; la fase final se arma a mano desde 'Agregar cruce manual' (ver Sección 6.2).
- **Estadísticas individuales de vóley y básquet ('Anotadores'):** las pantallas existen pero muestran 'módulo en preparación' — no hay captura de puntos por jugador para esos deportes, solo el marcador global del equipo.
- **Modalidades volley_mixto y basket_3x3:** aparecen como opción seleccionable en el formulario, pero no tienen reglas propias más allá de los valores por defecto de jugadores en cancha/periodos; no hay diferenciación real de reglamento frente a volley_sala/basket_5.
- **Planillas PDF por deporte:** el título de la planilla usa el deporte correcto, pero el diseño interno (columnas de goles y tarjetas) es el mismo para fútbol, vóley y básquet — no hay una planilla orientada a sets o a faltas todavía.

7.3 Inconsistencias y código huérfano detectados

- **Módulo de Auditoría siempre vacío:** auditoria_screen.dart lee campeonatos/{id}/auditoria, pero ningún service del proyecto escribe en esa colección. El sistema sí registra auditoría real, pero en otras colecciones (historial_cambios de jugador, historial_planilla de equipo).
- **HistorialJugadorScreen inalcanzable:** la pantalla y su lógica están completas y funcionales, pero ninguna otra pantalla del proyecto navega hacia ella — no hay forma de abrirla desde la interfaz actual.
- **GrupoModel sin usar:** el modelo y su archivo (grupo_model.dart) no se referencian desde ningún service ni pantalla. La fase de grupos real se resuelve con un simple campo String groupId en cada partido, no con este modelo ni con una colección de grupos.
- **CampeonatoConfig.permitePenales / permiteProrroga sin leer:** se escriben al crear el campeonato pero ningún archivo del proyecto los vuelve a leer; ResultadoService recalcula la necesidad de penales/prórroga de forma independiente a partir de permiteEmpate y tipoCampeonato.
- **CampeonatoModel.reglasDesempate sin aplicar:** se guarda una lista distinta por deporte pero el ordenamiento real de la tabla de posiciones nunca la lee (ver Sección 6.1).

- **Acceso a Firestore inconsistente:** la mayoría de las colecciones pasan por un service dedicado, pero 'goles' y 'tarjetas' se consultan directamente con `Firestore.instance` desde dentro de las pantallas (`tabla_posiciones_screen.dart` y `pdfs_screen.dart`) en vez de exponerse desde `ResultadoService`.
- **Nombre de pantalla engañoso:** `TablaPosicionesScreen` (módulo 'Tabla' del panel administrativo) no muestra la tabla de posiciones — muestra el historial de partidos jugados. La tabla de posiciones real solo existe en la vista pública (`ChampionshipDetailScreen`).
- **Credenciales de Firebase en el código fuente:** `firebase_options.dart` contiene las claves de los 5 targets embebidas en texto plano (comportamiento estándar de `FlutterFire`, no un bug de programación, pero queda expuesto en el repositorio — ver Sección 3.2).

Fin del análisis. Todo el contenido de este documento se basó exclusivamente en la lectura del código fuente presente en `lib/` al momento de su generación; no se modificó ningún archivo del proyecto.